Using Epistemic Observability in Agentic Systems for Combating Hallucinations

Tony Mason
The University of British Columbia

Vaastav Anand
Max Planck Institute for Software Systems

Abstract

LLMs are increasingly deployed as components of production systems, introducing a fundamental challenge: generated outputs may be hallucinated, while verification incurs computational and monetary cost. Consequently, systems cannot afford to verify every generation equally and must instead decide how to allocate a limited verification budget. This raises a fundamental question: what additional reliability does each increment of verification investment buy?

We argue that text-only observation, even with bounded supervision, is structurally insufficient for reliably assessing the epistemic validity of model outputs, regardless of model scale or training procedure. To address this limitation, we introduce epistemic observability, an interface that exposes internal computational telemetry alongside generated text as these signals they provide systems with richer evidence for estimating hallucination risk and prioritizing verification effort. We present a four-tier epistemic observability hierarchy that characterizes the trade-off between verification cost and the amount of epistemic information available to downstream systems. As a first realization of this framework, we design a tensor-generation interface that exposes Tier 1 epistemic telemetry and demonstrate that tensor-guided judges consistently outperform text-only baselines across all verification budgets, improving accuracy by 2.5–3.9 percentage points on average across four model architectures.

1 Introduction

Systems incorporating language model components face a verification problem. The components generate text by sampling from learned probability distributions, and some fraction of that text is fabricated: fluent, confident, and wrong. LLM-based systems today are rife with "hallucinations". We use the term *fabrication* throughout this paper to emphasize that the system's default behavior under coherence optimization is reasonable and not a malfunction to be patched. Xu et al. [18] proves that fabrication is inevitable for LLMs over the class of computable functions.

Consequently, every system that deploys a language model component must allocate a budget: how much of the output to verify, which outputs to prioritize, and what signals to use for triage. For example, checking a citation against a database costs latency; running a second model as a judge costs compute. Verification cost varies by content type: checking a citation is cheap; assessing whether a summary faithfully represents its source is expensive.

The key issue is that the current text-only interface to language models does not expose the model's internal computation, which produces signals that distinguish grounded generation from fabrication. The model may know it is fabricating but currently the supervisor cannot tell if the model is fabricating just from the text output.

This observability gap has direct consequences for verification cost. A supervisor receiving only text must spend its verification budget reconstructing, from the output alone, what the model's own computation already knew. The budget is consumed by reconstruction rather than concentrated on the cases where verification is most needed.

Our key insight is that language models carry internal signals that distinguish grounded generation from fabrication. These signals are byproducts of the computation itself: *telemetry* that the model cannot separately control under standard training. Unlike the model's *testimony* (the text it chooses to output), telemetry is harder to fake.

We introduce *epistemic observability*: an interface that exposes this telemetry alongside the standard text output. Unlike interpretability, which seeks to explain why a model activates particular neurons, or explainability, which generates post-hoc rationales for its outputs, epistemic observability surfaces signals produced as a byproduct of inference that help estimate whether the generated content is grounded in the model's knowledge or is likely to be fabricated.

To support a range of application requirements, we propose a multi-tier epistemic observability interface, where each successive tier exposes richer and more informative telemetry about the generation process. These tiers provide a principled trade-off between observability and computational cost, enabling developers to select the level of epistemic insight that best matches their system's reliability requirements and resource budget. For instance, a creative brainstorming assistant may require only Tier 0 observability, whereas a medical decision-support system may justify the additional cost of higher observability tiers to better identify and validate potentially hallucinated content. In this way, the observability hierarchy makes explicit what additional epistemic information each level of investment provides.

As a proof of concept, we design a tensor-generation interface that exposes Tier 1 epistemic telemetry, which includes the per-token entropy, and demonstrate that tensor-guided judges consistently outperform text-only baselines across all verification budgets, improving accuracy by 2.5–3.9 percentage points on average across four model architectures.

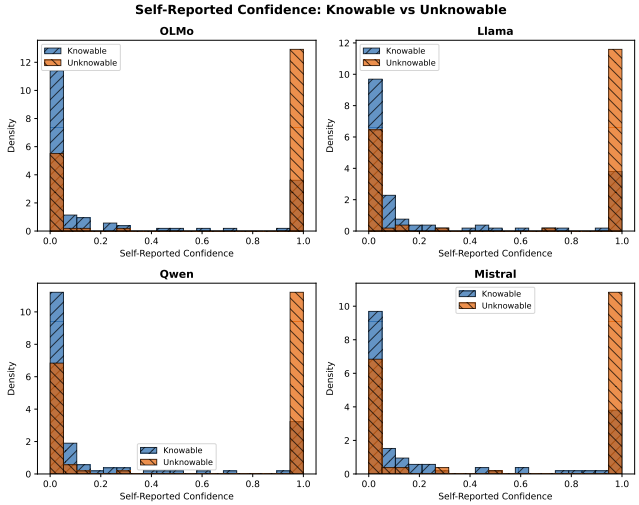

Figure 1. Self-reported confidence is inverted: all four models report higher confidence on unknowable queries

Type	Description
Adversarial Truth	True but implausible
Shattered Lie	No training-data grounding
Deceived Lie	Pattern-matches plausible language
Confused Truth	Counterintuitive fact

Table 1. Hallucination Types that span the verification space.

2 Background & Motivation

A language model maps input tokens to output tokens, producing internal signals (entropy, attention patterns, logprobabilities) as computational byproducts. The training corpus contains both truth-tracking institutional text and misinformation. This results in four canonical hallucination types, shown in Table 1.

A language model trained on this corpus inherits these mixed regularities. Base models (pre-fine-tuning) fabricate substantially on factual benchmarks because they are not honest by default. But they also encode rich internal representations of concepts like correctness and truthfulness, which can be elicited under the right conditions [3]. The base model's relationship to truth is complex: neither reliably honest nor uniformly deceptive. The question for systems engineers is not whether the model is honest, but what it costs to verify when it is not.

2.1 Testimony & Telemetry

Architecture primer. A transformer-based language model generates text one *token* (subword unit) at a time, autoregressively: at each step it produces a probability distribution over its vocabulary, selects a token, and conditions the next step on all prior output. Each step passes through a stack of layers producing *attention patterns* (weighted relevance scores between positions) and intermediate representations. The *entropy* of the output distribution, the attention patterns, and the *log-probabilities* of selected tokens are byproducts of this computation: telemetry the model cannot control.

Existing models operate with a text-only output interface. The text is the testimony, whereas the telemetry are measurements of the computation that produced the output. The model can control its testimony; under current training objectives, it cannot independently control its telemetry. From a cost perspective: testimony is cheap to produce and cheap to inspect but unreliable; telemetry is more expensive to export and process but, under current training regimes, harder to game because it is a byproduct of the computation rather than a product of the model's optimization objective. This decoupling is an empirical property of existing training pipelines, not an architectural guarantee: whether adversarial training could learn to couple telemetry to the optimization objective is an open question.

2.2 The Text Channel Cannot Self-Verify

Current models cannot be trusted to honestly report their own epistemic state through the text channel. To demonstrate the inadequacy of text-only verification, we use a taxonomy of query types: *knowable* queries (facts the model should have stable representations for) versus *unknowable* queries (fabrication prompts, future events, private information) and report each model's self-confidence score for the self-reported confidence baseline, the cheapest verification strategies available to a bounded supervisor. In this baseline, the supervisor asks the model "How confident are you?" after generation and tests whether the model can introspect and report its epistemic state through text.

Figure 1 shows that self-reported confidence is *inverted*: all four models report higher confidence on unknowable queries than on knowable ones, yielding AUC 0.28–0.36 across architectures (all below random). A naive user who asks "how confident are you?" receives a number that anticorrelates with actual epistemic grounding, while an adversary could exploit this inversion to make fabricated outputs appear maximally confident. Under realistic mixed-objective incentives and bounded oversight, the text-only interface is insufficient: a supervisor cannot distinguish truthful uncertainty from confident fabrication.

3 Epistemic Observability

We defined epistemic observability as the ability of a language model to expose telemetry about its epistemic state during inference, enabling downstream systems to reason about the reliability of a generated response. Rather than relying solely on the model's generated text, epistemic observability provides additional signals produced as byproducts of the inference process, such as decoding statistics, internal representations, or computational traces. These signals provide evidence about how trustworthy the generation is and whether it is likely to be fabricated.

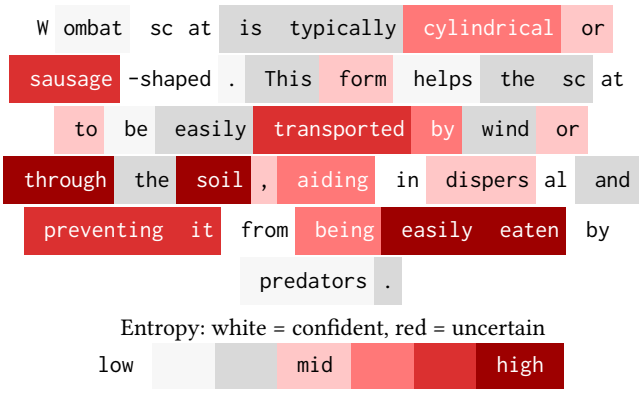

Figure 2. Generated output along with its entropy trace for the query *What shape is wombat scat?*

3.1 Observability Tiers

We present a four-tier epistemic observability hierarchy that characterizes the trade-off between verification cost and the amount of epistemic information available to downstream systems for assessing reliability of the generated text.

Tier 0: Output Observability. Tier 0 exposes only the generated text, treating the language model as a black box. This is the interface available through most commercial APIs and forms the basis of existing validation techniques such as retrieval-based verification, secondary LLM judges, symbolic validators, and application-specific consistency checks. Since no internal telemetry is available, verification must rely entirely on properties of the generated output, often requiring expensive external computation.

Tier 1: Confidence Observability. Tier 1 augments the generated text with lightweight telemetry produced during decoding. Examples include token probabilities, logit distributions, entropy [9, 17], confidence scores, decoding statistics, and other signals that can be extracted with negligible overhead. These signals provide a coarse estimate of the model's uncertainty [15], allowing systems to identify portions of the output that may warrant additional validation. While inexpensive to obtain, confidence signals are often imperfectly calibrated and may fail to distinguish confidently stated hallucinations from grounded generations.

Tier 2: Representation Observability. Tier 2 exposes information derived from the model's internal representations. Rather than observing only the final decoding probabilities, systems may access hidden-state embeddings, layer activations, attention statistics, or learned probes [1, 2] that estimate properties of the latent epistemic state. Simple probes can catch deviant model behaviors [4, 5, 11]. Recent techniques such as semantic entropy probes [7, 8], hidden-state classifiers [19], and activation-based uncertainty estimators demonstrate that these representations often contain substantially richer information about generation reliability than logits alone. Representation observability enables more accurate prioritization of verification effort while remaining significantly cheaper than exhaustive external validation.

Tier 3: Mechanistic Observability. Tier 3 exposes telemetry obtained through mechanistic analysis of the model's computation. Mechanistic techniques identify the causal computations that produce a generation by tracing information flow through neurons, attention heads, and latent features [16]. This enables systems to distinguish between different failure modes and provides insight into *why* a hallucination occurred rather than merely detecting that one occurred. They can reveal whether a response is supported by robust internal computation or arises from fragile or spurious reasoning pathways. Mechanistic observability provides the highest-fidelity view of a model's epistemic process at the highest cost.

Choosing an Observability Tier. The appropriate tier depends on the reliability requirements and resource constraints of the target application. Creative writing assistants, brainstorming tools, and conversational agents may operate effectively with Tier 0 or Tier 1 observability, where occasional hallucinations are acceptable and verification budgets are limited. In contrast, high-assurance systems such as medical decision support, legal analysis, or scientific assistants may benefit from Tier 2 or Tier 3 observability, where richer epistemic telemetry enables verification resources to be focused on the portions of the generation most likely to contain unsupported or fabricated content. Rather than prescribing a single solution, the observability hierarchy provides a principled framework for reasoning about the trade-off between verification cost and the amount of epistemic information available to guide that verification.

3.2 Tier 1 Tensor Interface

We propose a minimal tensor interface for Tier 1 observability. We claim the tensor reveals whether the model is operating in *grounded mode* (retrieving from stable representations) versus *fabrication mode* (generating without anchoring). The tensor interface is a generation interface that returns: (i) **Text**: The linearized response string (standard interface output). (ii) **Entropy trace**: Per-token entropy of the output distribution during generation. (iii) **Attention summary**: Statistics of attention patterns (concentration, self-attention ratio) from deeper processing layers.

This interface exports telemetry that the model cannot separately control under current training objectives. The model can lie in its text, but it cannot present a different inference computation than the one it actually performed. Independent work on persona monitoring confirms this asymmetry: activation projections reveal that models adopt distinct internal configurations for different epistemic conditions even when producing textually similar outputs [10]. This asymmetry is the foundation of the adversarial robustness argument: under current training objectives, computational signals (entropy, attention geometry) cannot be independently controlled by the model without changing the generation process itself,

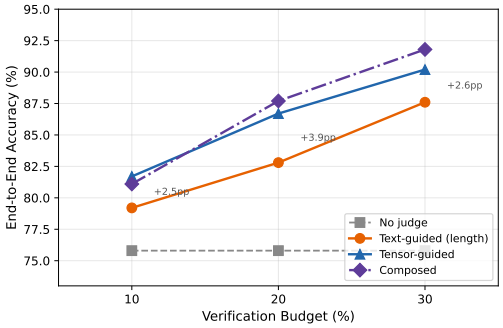

Figure 3. End-to-end accuracy vs. verification budget for four judge conditions, averaged across four architectures.

unlike behavioral signals (response length, hedging patterns) that can be adapted through training without altering what the model knows or does not know.

What the tensor provides. The entropy trace provides a falsifiability signal. If a model claims certainty while its entropy trace shows high uncertainty, the mismatch is detectable. The attention summary provides a complementary signal: fabrications tend to show more dispersed attention patterns than grounded responses.

What the tensor does not provide. The tensor interface is not a lie detector. It does not prove that low-entropy outputs are true or that high-entropy outputs are false. It provides *one additional signal* that, combined with other verification methods, can improve epistemic assessment.

Implementation. The tensor interface requires no architectural changes to current transformers. Generation APIs already support returning logits; the entropy trace is a simple transformation. Attention patterns are available with `output_attentions=True` in standard frameworks. We provide a reference implementation supporting OLMo models that demonstrates the interface is practical. The overhead is modest: storing per-token entropy and summarizing attention patterns adds approximately 2–7% latency depending on signal set, compared to generation itself.

Example Entropy Trace. Figure 2 shows the generated output for the query *What shape is wombat scat?* along with its entropy trace. We color code per-token entropy: blue indicates low entropy (confident generation), orange indicates moderate entropy, and brown indicates high entropy (uncertain generation). The entropy enhanced output shows where in the output generation did the generation proceed with high confidence (low entropy, suggesting retrieval of stored pattern) and where did it proceed with high uncertainty (high entropy, suggesting fabrication).

4 Evaluation

We evaluate the return on verification investment across four strategies operating at three budget levels (the fraction of outputs the judge may verify and intervene on).

Experimental Setup. We construct a balanced set of 200 queries: 100 knowable (verifiable facts spanning multiple subjects) and 100 unknowable (fabrication prompts for nonexistent people, fictional syndromes, future events, and nonexistent citations). Ground truth labels are established by construction. We run all queries on four architectures: OLMo-3 7B [13], Llama-3.1 8B [12], Qwen3 4B [14], and Mistral 7B [6], using instruct-tuned variants to ensure consistent question-answering behavior across all models.

Conditions. We evaluate each model under four conditions sharing a fixed verification budget: (i) **No judge.** Raw model output; the unverified baseline. (ii) **Text-guided judge (length).** Selects outputs for verification using response word count, a text-channel signal available without any tensor interface. (iii) **Tensor-guided judge.** Selects outputs using per-token entropy and attention summary statistics. (iv) **Composed judge.** Tensor-guided selection for general queries; bounded lookup judge for citation queries where tensor signals are known to invert. We evaluate each condition at 10%, 20%, and 30% verification budgets and measure end-to-end accuracy.

Figure 3 shows that **tensor-guided verification outperforms the text baseline at every budget level**: +2.5pp at 10%, +3.9pp at 20%, +2.6pp at 30%. Each line maps an investment strategy to its return. No judge is the zero-investment baseline, Text-guided uses response word count, a text-channel signal available at zero marginal cost; Tensor-guided uses per-token entropy, which discriminates among outputs of similar length, reaching cases the text channel cannot. Per-model results confirm the pattern is consistent across all four tested architectures. Tensor-guided verification improves accuracy for every model tested: OLMo (69.5% → 75.1% at 10%), Llama (82.5% → 88.1%), Qwen (87.0% → 91.7%), and Mistral (64.0% → 72.0%).

Composed judges and complementary failure modes. At 30% budget, the composed judge (91.8%) outperforms the tensor-guided judge alone (90.2%). The advantage is small in aggregate but structurally significant: it comes entirely from citation queries where entropy signals *invert*. Fabricated citations produce lower entropy than real ones because the model generates plausible fakes more fluently than it retrieves specific bibliographic details.

5 Conclusion

Systems incorporating language model components must decide how much resources to spend on verification and where to spend it. We introduce the notion of epistemic observability to provide the cost surface for making that decision. We present different tiers of epsitemic observability for providing different levels of verification, and build a tensor interface at the logit-based observability tier to show how it can reduce the budget required for reducing hallucinations and improving accuracy.

References

[1] Guillaume Alain and Yoshua Bengio. 2016. Understanding intermediate layers using linear classifier probes. *arXiv preprint arXiv:1610.01644* (2016).

[2] Yonatan Belinkov. 2022. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics* 48, 1 (2022), 207–219.

[3] Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. 2023. Discovering Latent Knowledge in Language Models Without Supervision. In *International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=ETKGuby0hcs

[4] Nicholas Goldowsky-Dill, Bilal Chughtai, Stefan Heimersheim, and Marius Hobbhahn. 2025. Detecting strategic deception using linear probes. *arXiv preprint arXiv:2502.03407* (2025).

[5] Jiatong Han, Neil Band, Muhammed Razzak, Jannik Kossen, Tim GJ Rudner, and Yarin Gal. 2025. Simple factuality probes detect hallucinations in long-form natural language generation. *Findings of the Association for Computational Linguistics: EMNLP* (2025), 16209–16226.

[6] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. arXiv:2310.06825 [cs.CL] https://arxiv.org/abs/2310.06825

[7] Jannik Kossen, Jiatong Han, Muhammed Razzak, Lisa Schut, Shreshth Malik, and Yarin Gal. 2024. Semantic entropy probes: Robust and cheap hallucination detection in llms. *arXiv preprint arXiv:2406.15927* (2024).

[8] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. 2023. Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation. arXiv:2302.09664 [cs.CL] https://arxiv.org/abs/2302.09664

[9] Chen Ling, Xujiang Zhao, Wei Cheng, Yanchi Liu, Yiyou Sun, Xuchao Zhang, Mika Oishi, Takao Osaki, Katsushi Matsuda, Jie Ji, et al. 2024. Uncertainty decomposition and quantification for in-context learning of large language models. *arXiv preprint arXiv:2402.10189* (2024).

[10] Christina Lu, Jack Gallagher, Jonathan Michala, Kyle Fish, and Jack Lindsey. 2026. The Assistant Axis: Situating and Stabilizing the Default Persona of Language Models. arXiv:2601.10387 [cs.CL] https://arxiv.org/abs/2601.10387

[11] Monte MacDiarmid, Timothy Maxwell, Nicholas Schiefer, Jesse Mu, Jared Kaplan, David Duvenaud, Sam Bowman, Alex Tamkin, Ethan Perez, Mrinank Sharma, Carson Denison, and Evan Hubinger. 2024. *Simple probes can catch sleeper agents.* https://www.anthropic.com/news/probes-catch-sleeper-agents

[12] Meta AI. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI] https://arxiv.org/abs/2407.21783

[13] Team Olmo, :, Allyson Ettinger, Amanda Bertsch, Bailey Kuehl, David Graham, David Heineman, Dirk Groeneveld, Faeze Brahman, Finbarr Timbers, Hamish Ivison, Jacob Morrison, Jake Poznanski, Kyle Lo, Luca Soldaini, Matt Jordan, Mayee Chen, Michael Noukhovitch, Nathan Lambert, Pete Walsh, Pradeep Dasigi, Robert Berry, Saumya Malik, Saurabh Shah, Scott Geng, Shane Arora, Shashank Gupta, Taira Anderson, Teng Xiao, Tyler Murray, Tyler Romero, Victoria Graf, Akari Asai, Akshita Bhagia, Alexander Wettig, Alisa Liu, Aman Rangapur, Chloe Anastasiades, Costa Huang, Dustin Schwenk, Harsh Trivedi, Ian Magnusson, Jaron Lochner, Jiacheng Liu, Lester James V. Miranda, Maarten Sap, Malia Morgan, Michael Schmitz, Michal Guerquin, Michael Wilson, Regan Huff, Ronan Le Bras, Rui Xin, Rulin Shao, Sam Skjonsberg, Shannon Zejiang Shen, Shuyue Stella Li, Tucker Wilde, Valentina Pyatkin, Will Merrill, Yapei Chang, Yuling Gu, Zhiyuan Zeng, Ashish Sabharwal, Luke Zettlemoyer, Pang Wei Koh, Ali Farhadi, Noah A. Smith, and Hannaneh Hajishirzi. 2025. Olmo 3. arXiv:2512.13961 [cs.CL] https://arxiv.org/abs/2512.13961

[14] Qwen Team. 2025. Qwen3 Technical Report. Alibaba Cloud technical report. https://qwenlm.github.io/blog/qwen3/

[15] Ola Shorinwa, Zhiting Mei, Justin Lidard, Allen Z Ren, and Anirudha Majumdar. 2025. A survey on uncertainty quantification of large language models: Taxonomy, open research challenges, and future directions. *Comput. Surveys* 58, 3 (2025), 1–38.

[16] Shriyank Somvanshi, Md Monzurul Islam, Amir Rafe, Anannya Ghosh Tusti, Arka Chakraborty, Anika Baitullah, Tausif Islam Chowdhury, Nawaf Alnawmasi, Anandi Dutta, and Subasish Das. 2026. Bridging the black box: a survey on mechanistic interpretability in AI. *Comput. Surveys* 58, 8 (2026), 1–35.

[17] Yijun Xiao and William Yang Wang. 2021. On hallucination and predictive uncertainty in conditional language generation. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*. 2734–2744.

[18] Ziwei Xu, Sanjay Jain, and Mohan Kankanhalli. 2024. Hallucination is Inevitable: An Innate Limitation of Large Language Models. arXiv:2401.11817 [cs.CL]

[19] Zhenliang Zhang, Xinyu Hu, Huixuan Zhang, Junzhe Zhang, and Xiaojun Wan. 2025. ICR probe: Tracking hidden state dynamics for reliable hallucination detection in LLMs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 17986–18002.